

LAPORAN PRAKTIKUM STRUKTUR DATA

“Algoritma Traversal pada Struktur Data Pohon Biner dan Graf”

disusun Oleh:

Az Zahrand Solichul Tajussalathin

NIM 2511532001

Dosen Pengampu: Dr. Wahyudi S.T M.T

Asisten Pratikum: Jovantri Immanuel Gulo



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

DAFTAR ISI	i
KATA PENGANTAR.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
2.1 Program Node.....	2
2.2 Program BTree	3
2.3 Program BtreeDriver	5
2.4 Program GraphTraversal.....	6
BAB III TUGAS	9
3.1 Program Sorting.....	9
3.2 Alasan Implementasi Algoritma	22
BAB IV KESIMPULAN	24
4.1 Ringkasan Hasil Praktikum	24
DAFTAR PUSTAKA.....	iii

KATA PENGANTAR

Puji syukur kehadiran Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum dengan judul “Algoritma Traversal pada Struktur Data Pohon Biner dan Graf” ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Struktur Data.

Ucapan terima kasih disampaikan kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran dalam bahasa pemrograman Java.

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Untuk data yang memiliki hubungan hierarkis atau relasi jaringan yang kompleks, diperlukan struktur data non-linear. Tree adalah struktur data hierarkis yang mensimulasikan struktur akar hingga daun, sangat berguna dalam sistem file atau hierarki organisasi. Di sisi lain, Graph merepresentasikan jaringan simpul yang saling terhubung tanpa aturan hierarki mutlak, yang menjadi fondasi utama dalam sistem navigasi peta.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep dan struktur dasar dari Binary Tree dan Graph.
2. Mengimplementasikan operasi dasar pada Binary Tree, termasuk penambahan simpul dan algoritma penelusuran Preorder, Inorder, serta Postorder.
3. Mengimplementasikan struktur Graph menggunakan Adjacency List

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini antara lain:

1. Meningkatkan kemampuan berpikir algoritmik dalam menangani struktur data yang bercabang dan saling terhubung.
2. Membekali pemahaman terkait teknik rekursi tingkat lanjut yang efisien untuk membedah kedalaman dari suatu struktur data non-linear.

BAB 2

PEMBAHASAN

2.1 Program Node

```
package pekan9_2511532001;

public class Node_2511532001 {
    int data_2001;
    Node_2511532001 left_2001;
    Node_2511532001 right_2001;
    public Node_2511532001 (int data_2001) {
        this.data_2001 = data_2001;
        left_2001 = null;
        right_2001 = null;
    }
    public void setLeft_2001 (Node_2511532001 node_2001) {
        if (left_2001 == null)
            left_2001 = node_2001;
    }
    public void setRight_2001 (Node_2511532001 node_2001) {
        if (right_2001 == null)
            right_2001 = node_2001;
    }
    public Node_2511532001 getLeft_2001() {
        return left_2001;
    }
    public Node_2511532001 getRight_2001() {
        return right_2001;
    }
    public int getData_2001() {
        return data_2001;
    }
    public void setData_2001(int data_2001) {
        this.data_2001 = data_2001;
    }
}

void printPreorder_2001 (Node_2511532001 node_2001) {
    if (node_2001 == null)
        return;
    System.out.print(node_2001.data_2001 + " ");
    printPreorder_2001(node_2001.left_2001);
    printPreorder_2001(node_2001.right_2001);
}

void printPostorder_2001 (Node_2511532001 node_2001) {
    if (node_2001 == null)
        return;
    printPostorder_2001(node_2001.left_2001);
    printPostorder_2001(node_2001.right_2001);
    System.out.print(node_2001.data_2001 + " ");
}
```

```

    }
    void printInorder_2001 (Node_2511532001 node_2001) {
        if (node_2001 == null)
            return;
        printInorder_2001(node_2001.left_2001);
        System.out.print(node_2001.data_2001 + " ");
        printInorder_2001(node_2001.right_2001);
    }
    public String print_2001() {
        return this.print_2001("", true, "");
    }
    public String print_2001(String prefix_2001, boolean isTail_2001,
String sb_2001) {
        if (right_2001 != null ) {
            right_2001.print_2001(prefix_2001 + (isTail_2001 ? "| ": " "),
false, sb_2001);
        }
        System.out.println( prefix_2001 + (isTail_2001 ? "\\-- ": "/-- ") +
data_2001);
        if (left_2001 != null) {
            left_2001.print_2001(prefix_2001 + (isTail_2001 ? " ": "| "),
true, sb_2001);
        }
        return sb_2001;
    }
}

```

Kelas Node berperan sebagai blueprint dalam pembentukan setiap simpul pada struktur data pohon biner. Kelas ini memiliki tiga komponen esensial: atribut data dengan tipe integer yang berfungsi menampung nilai simpul, ditambah dua pointer memori (left dan right) yang menunjuk ke cabang kiri dan cabang kanan. Tidak hanya menyimpan data, kelas ini juga mengelola logika penjelajahan pohon secara rekursif. Metode-metode seperti printPreorder, printInorder, dan printPostorder ditanamkan di dalam kelas sehingga setiap node bisa mengeksekusi dirinya sendiri serta sub-pohonnya secara mandiri. Selain itu, terdapat metode print yang dirancang khusus untuk menggambar struktur hierarki pohon ke terminal menggunakan pola garis.

2.2 Program BTree

```

package pekan9_2511532001;

public class BTree_2511532001 {
    private Node_2511532001 root_2001;
    private Node_2511532001 currentNode_2001;
    public BTree_2511532001 () {
        root_2001 = null;
    }
}

```

```

public boolean search_2001(int data_2001) {
    return search_2001(root_2001, data_2001);
}
private boolean search_2001(Node_2511532001 node_2001, int data_2001) {
    if (node_2001.getData_2001() == data_2001)
        return true;
    if (node_2001.getLeft_2001() != null)
        if (search_2001(node_2001.getLeft_2001(), data_2001))
            return true;
    if (node_2001.getRight_2001() != null)
        if (search_2001(node_2001.getRight_2001(), data_2001))
            return true;
    return false;
}
public void printInorder_2001 () {
    root_2001.printInorder_2001(root_2001);
}
public void printPreorder_2001 () {
    root_2001.printPreorder_2001(root_2001);
}
public void printPostorder_2001 () {
    root_2001.printPostorder_2001(root_2001);
}
public Node_2511532001 getRoot_2001() {
    return root_2001;
}
public boolean isEmpty_2001() {
    return root_2001 == null;
}

public int countNodes_2001() {
    return countNodes_2001(root_2001);
}
private int countNodes_2001(Node_2511532001 node_2001) {
    int count_2001 = 1;
    if (node_2001 == null) {
        return 0;
    } else {
        count_2001 += countNodes_2001(node_2001.getLeft_2001());
        count_2001 += countNodes_2001(node_2001.getRight_2001());
        return count_2001;
    }
}
public void print_2001() {
    root_2001.print_2001();
}
public Node_2511532001 getCurrentNode_2001() {
    return currentNode_2001;
}
public void setCurrentNode_2001(Node_2511532001 node_2001) {
    this.currentNode_2001 = node_2001;
}
public void setRoot_2001(Node_2511532001 root_2001) {

```

```

        this.root_2001 = root_2001;
    }
}

```

Kelas BTree berperan sebagai manager dari struktur pohon secara keseluruhan. Kelas ini tidak menyimpan data mentah, melainkan hanya merekam titik awal referensi pohon melalui variabel root. Konsep enkapsulasi digunakan di kelas ini; ketika pengguna memanggil metode `printlnorder` pada objek BTree, metode tersebut di belakang layar akan memerintahkan elemen root untuk mengeksekusi rekursinya. Selain itu, BTree menggunakan metode pencarian elemen (`search`) dan metode penghitungan jumlah total simpul (`countNodes`) dengan cara menelusuri penambahan nilai dari setiap cabang secara rekursif dari atas ke bawah.

2.3 Program BtreeDriver

```

package pekan9_2511532001;

public class BtreeDriver_2511532001 {
    public static void main(String[] args) {
        // Membuat pohon
        BTree_2511532001 tree_2001 = new BTree_2511532001();
        System.out.print("Jumlah Simpul awal pohon: ");
        System.out.println(tree_2001.countNodes_2001());

        // Menambahkan simpul data 1
        Node_2511532001 root_2001 = new Node_2511532001(1);

        //Menjadikan simpul 1 sebagai root
        tree_2001.setRoot_2001(root_2001);
        System.out.print("Jumlah simpul jika hanya ada root");
        System.out.print(tree_2001.countNodes_2001());
        Node_2511532001 node2 = new Node_2511532001(2);
        Node_2511532001 node3 = new Node_2511532001(3);
        Node_2511532001 node4 = new Node_2511532001(4);
        Node_2511532001 node5 = new Node_2511532001(5);
        Node_2511532001 node6 = new Node_2511532001(6);
        Node_2511532001 node7 = new Node_2511532001(7);
        Node_2511532001 node8 = new Node_2511532001(8);
        Node_2511532001 node9 = new Node_2511532001(9);
        root_2001.setLeft_2001(node2);
        node2.setLeft_2001(node4);
        node2.setRight_2001(node5);
        node4.setRight_2001(node8);
        root_2001.setRight_2001(node3);
        node3.setLeft_2001(node6);
        node3.setRight_2001(node7);
        node6.setLeft_2001(node9);
    }
}

```



```

        // Set root
        tree_2001.setCurrentNode_2001(tree_2001.getRoot_2001());
        System.out.println("menampilkan simpul terakhir: ");
        System.out.println(tree_2001.getCurrentNode_2001().getData_2001());
        System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
        System.out.println(tree_2001.countNodes_2001());
        System.out.println("InOrder: ");
        tree_2001.printInorder_2001();
        System.out.println("\nPreorder: ");
        tree_2001.printPreorder_2001();
        System.out.println("\nPostorder : ");
        tree_2001.printPostorder_2001();
        System.out.println("\nMenampilkan simpul dalam bentuk pohon");
        tree_2001.print_2001();
    }
}

```

Program BtreeDriver merupakan class driver untuk membangun pohon biner secara manual. Berbeda dengan Binary Search Tree (BST) yang menyusun data secara otomatis berdasarkan nilai angka, program ini mengaitkan node satu per satu menggunakan metode setLeft dan setRight. Objek node1 hingga node9 dirangkai membentuk sebuah struktur pohon kustom, yang kemudian dievaluasi menggunakan metode print untuk memvisualisasikan bentuk cabangnya langsung di terminal, serta dieksekusi penelusurannya menggunakan Preorder, Inorder, dan Postorder untuk melihat perbedaan hasil urutan cetak datanya.

2.4 Program GraphTraversal

```

package pekan9_2511532001;

import java.util.*;

public class GraphTraversal_2511532001 {
    private Map<String, List<String>> graph_2001 = new HashMap<>();

    // Menambahkan edge (graf tak berarah)
    public void addEdge_2001(String node1_2001, String node2_2001) {
        graph_2001.putIfAbsent(node1_2001, new ArrayList<>());
        graph_2001.putIfAbsent(node2_2001, new ArrayList<>());
        graph_2001.get(node1_2001).add(node2_2001);
        graph_2001.get(node2_2001).add(node1_2001);
    }

    // Menampilkan graf awal
    public void printGraph_2001() {
        System.out.println("Graf Awal (Adjacency List):");
        for (String node_2001 : graph_2001.keySet()) {

```

```

        System.out.print(node_2001 + " -> ");
        List<String> neighbors_2001 = graph_2001.get(node_2001);
        System.out.println(String.join(" ", neighbors_2001));
    }
    System.out.println();
}

// DFS rekursif
public void dfs_2001(String start_2001) {
    Set<String> visited_2001 = new HashSet<>();
    System.out.println("Penelusuran DFS:");
    dfsHelper_2001(start_2001, visited_2001);
    System.out.println();
}

private void dfsHelper_2001(String current_2001, Set<String>
visited_2001) {
    if (visited_2001.contains(current_2001)) return;

    visited_2001.add(current_2001);
    System.out.print(current_2001 + " ");

    for (String neighbor_2001 : graph_2001.getDefault(current_2001,
new ArrayList<>())) {
        dfsHelper_2001(neighbor_2001, visited_2001);
    }
}

// BFS iteratif
public void bfs_2001(String start_2001) {
    Set<String> visited_2001 = new HashSet<>();
    Queue<String> queue_2001 = new LinkedList<>();

    queue_2001.add(start_2001);
    visited_2001.add(start_2001);

    System.out.println("Penelusuran BFS:");
    while (!queue_2001.isEmpty()) {
        String current_2001 = queue_2001.poll();
        System.out.print(current_2001 + " ");

        for (String neighbor_2001 :
graph_2001.getDefault(current_2001, new ArrayList<>())) {
            if (!visited_2001.contains(neighbor_2001)) {
                queue_2001.add(neighbor_2001);
                visited_2001.add(neighbor_2001);
            }
        }
    }
    System.out.println();
}

// Main

```

```

    public static void main(String[] args_2001) {
        GraphTraversal_2511532001 graph_2001 = new
GraphTraversal_2511532001();

        // Contoh graf: A-B, A-C, B-D, B-E
        graph_2001.addEdge_2001("A", "B");
        graph_2001.addEdge_2001("A", "C");
        graph_2001.addEdge_2001("B", "D");
        graph_2001.addEdge_2001("B", "E");

        // Cetak graf awal
        System.out.println("Graf Awal adalah: ");
        graph_2001.printGraph_2001();

        // Lakukan penelusuran
        graph_2001.dfs_2001("A");
        graph_2001.bfs_2001("A");
    }
}

```

Program GraphTraversal mendemonstrasikan Graph tak berarah menggunakan Adjacency List berbasis HashMap dan List. Metode addEdge berfungsi memastikan setiap simpul saling mencatat koneksi mereka. Program ini mengimplementasikan dua algoritma penelusuran utama: Depth First Search (DFS) yang menelusuri jalur sedalam mungkin secara rekursif melalui dfsHelper, serta Breadth First Search (BFS) yang mengeksplorasi simpul secara melebar per level menggunakan bantuan queue. Dalam implementasinya, kedua algoritma ini wajib memanfaatkan objek HashSet (visited) untuk melacak simpul yang telah dikunjungi. Penandaan ini sangat krusial untuk mencegah terjadinya eksekusi infinite loop apabila terdapat jalur memutar (cyclic) pada arsitektur graf tersebut.

BAB 3

TUGAS

3.1 Program Sorting

```
package pekan9_2511532001;

import javax.swing.*;
import java.awt.*;
import java.util.*;
import java.util.List;

public class PetaKampus_2511532001 extends JFrame {

    // Struktur Data Graph
    private Map<String, List<String>> graph_2001;
    private Map<String, Point> nodeCoords_2001;

    // Variabel state untuk visualisasi
    private List<String> visitedNodes_2001;
    private List<String> pathNodes_2001;
    private int nodesExplored_2001 = 0;

    // Komponen GUI
    private JComboBox<String> startCombo_2001;
    private JComboBox<String> goalCombo_2001;
    private JTextArea resultArea_2001;
    private GraphPanel_2001 panelGraph_2001;

    public PetaKampus_2511532001() {
        setTitle("Pencarian Jalur Kampus Unand - BFS & DFS");
        setSize(800, 600);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        initGraphData_2001();

        // Panel Atas (Kontrol)
        JPanel controlPanel_2001 = new JPanel(new FlowLayout());

        String[] nodes_2001 = nodeCoords_2001.keySet().toArray(new
String[0]);
        Arrays.sort(nodes_2001); // Urutkan alfabetis untuk combobox

        startCombo_2001 = new JComboBox<>(nodes_2001);
        goalCombo_2001 = new JComboBox<>(nodes_2001);

        JButton btnBFS_2001 = new JButton("BFS");
        btnBFS_2001.setBackground(new Color(144, 238, 144));
    }
}
```

```

        JButton btnDFS_2001 = new JButton("DFS");
        btnDFS_2001.setBackground(new Color(255, 204, 102));

        JButton btnReset_2001 = new JButton("RESET");
        btnReset_2001.setBackground(new Color(255, 102, 102));
        btnReset_2001.setForeground(Color.WHITE);

        controlPanel_2001.add(new JLabel("Lokasi Awal:"));
        controlPanel_2001.add(startCombo_2001);
        controlPanel_2001.add(new JLabel("Lokasi Tujuan:"));
        controlPanel_2001.add(goalCombo_2001);
        controlPanel_2001.add(btnBFS_2001);
        controlPanel_2001.add(btnDFS_2001);
        controlPanel_2001.add(btnReset_2001);

        // Panel Tengah (Visualisasi Graph)
        panelGraph_2001 = new GraphPanel_2001();
        panelGraph_2001.setBackground(Color.WHITE);

panelGraph_2001.setBorder(BorderFactory.createTitledBorder("VISUALISASI
GRAPH"));

        // Panel Bawah (Hasil)
        resultArea_2001 = new JTextArea(6, 50);
        resultArea_2001.setEditable(false);
        resultArea_2001.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollResult_2001 = new JScrollPane(resultArea_2001);
        scrollResult_2001.setBorder(BorderFactory.createTitledBorder("Hasil
Pencarian"));

        add(controlPanel_2001, BorderLayout.NORTH);
        add(panelGraph_2001, BorderLayout.CENTER);
        add(scrollResult_2001, BorderLayout.SOUTH);

        // Event Listeners
        btnBFS_2001.addActionListener(e -> BFS_2001());
        btnDFS_2001.addActionListener(e -> DFS_2001());
        btnReset_2001.addActionListener(e -> resetGraph_2001());

        resetGraph_2001(); // Inisialisasi awal
    }

    private void initGraphData_2001() {
        graph_2001 = new HashMap<>();
        nodeCoords_2001 = new HashMap<>();

        // Inisialisasi 10 Node beserta koordinat GUI-nya
        nodeCoords_2001.put("Gerbang", new Point(100, 200));
        nodeCoords_2001.put("Masjid", new Point(250, 300));
        nodeCoords_2001.put("Asrama", new Point(250, 100));
        nodeCoords_2001.put("Rektorat", new Point(250, 200));
        nodeCoords_2001.put("Perpus", new Point(450, 100));
    }

```

```

nodeCoords_2001.put("PKM", new Point(450, 300));
nodeCoords_2001.put("Fak_Ekonomi", new Point(450, 200));
nodeCoords_2001.put("Fak_Hukum", new Point(650, 100));
nodeCoords_2001.put("FMIPA", new Point(650, 200));
nodeCoords_2001.put("FTI", new Point(650, 300));

for (String node : nodeCoords_2001.keySet()) {
    graph_2001.put(node, new ArrayList<>());
}

// Inisialisasi 15 Edge
addEdge_2001("Gerbang", "Rektorat");
addEdge_2001("Gerbang", "Masjid");
addEdge_2001("Gerbang", "Asrama");
addEdge_2001("Rektorat", "Fak_Ekonomi");
addEdge_2001("Rektorat", "Perpus");
addEdge_2001("Rektorat", "PKM");
addEdge_2001("Masjid", "PKM");
addEdge_2001("Asrama", "Perpus");
addEdge_2001("Perpus", "Fak_Hukum");
addEdge_2001("Perpus", "Fak_Ekonomi");
addEdge_2001("PKM", "Fak_Ekonomi");
addEdge_2001("PKM", "FTI");
addEdge_2001("Fak_Ekonomi", "FMIPA");
addEdge_2001("Fak_Hukum", "FMIPA");
addEdge_2001("FMIPA", "FTI");
}

private void addEdge_2001(String u, String v) {
    graph_2001.get(u).add(v);
    graph_2001.get(v).add(u); // Undirected graph
}

// Method Wajib: BFS
private void BFS_2001() {
    resetState_2001();
    String start_2001 = (String) startCombo_2001.getSelectedItem();
    String goal_2001 = (String) goalCombo_2001.getSelectedItem();

    Queue<String> queue_2001 = new LinkedList<>();
    Map<String, String> parentMap_2001 = new HashMap<>();

    queue_2001.add(start_2001);
    visitedNodes_2001.add(start_2001);
    parentMap_2001.put(start_2001, null);

    boolean found_2001 = false;

    while (!queue_2001.isEmpty()) {
        String current_2001 = queue_2001.poll();
        nodesExplored_2001++;

        if (current_2001.equals(goal_2001)) {

```

```

        found_2001 = true;
        break;
    }

    for (String neighbor : graph_2001.get(current_2001)) {
        if (!visitedNodes_2001.contains(neighbor)) {
            visitedNodes_2001.add(neighbor);
            parentMap_2001.put(neighbor, current_2001);
            queue_2001.add(neighbor);
        }
    }
}

if (found_2001) {
    reconstructPath_2001(parentMap_2001, goal_2001);
}
displayPath_2001("BFS");
displayGraph_2001();
}

// Method Wajib: DFS
private void DFS_2001() {
    resetState_2001();
    String start_2001 = (String) startCombo_2001.getSelectedItem();
    String goal_2001 = (String) goalCombo_2001.getSelectedItem();

    Stack<String> stack_2001 = new Stack<>();
    Map<String, String> parentMap_2001 = new HashMap<>();

    stack_2001.push(start_2001);
    parentMap_2001.put(start_2001, null);

    boolean found_2001 = false;

    while (!stack_2001.isEmpty()) {
        String current_2001 = stack_2001.pop();

        if (!visitedNodes_2001.contains(current_2001)) {
            visitedNodes_2001.add(current_2001);
            nodesExplored_2001++;

            if (current_2001.equals(goal_2001)) {
                found_2001 = true;
                break;
            }
        }

        // Push tetangga ke stack
        for (String neighbor : graph_2001.get(current_2001)) {
            if (!visitedNodes_2001.contains(neighbor)) {
                stack_2001.push(neighbor);
                parentMap_2001.put(neighbor, current_2001);
            }
        }
    }
}

```

```

    }
}

    if (found_2001) {
        reconstructPath_2001(parentMap_2001, goal_2001);
    }
    displayPath_2001("DFS");
    displayGraph_2001();
}

    private void reconstructPath_2001(Map<String, String> parentMap_2001,
String goal_2001) {
    String current_2001 = goal_2001;
    while (current_2001 != null) {
        pathNodes_2001.add(0, current_2001); // Insert at beginning to
reverse
        current_2001 = parentMap_2001.get(current_2001);
    }
}

// Method Wajib: displayPath
private void displayPath_2001(String algorithm_2001) {
    StringBuilder sb = new StringBuilder();
    sb.append("Hasil Pencarian ").append(algorithm_2001).append("
:
\n");

    if (pathNodes_2001.isEmpty()) {
        sb.append("Jalur : Tidak Ditemukan\n");
    } else {
        sb.append("Jalur : ").append(String.join(" -> ",
pathNodes_2001)).append("\n");
    }

    sb.append("Node Dikunjungi : ").append(String.join(" -> ",
visitedNodes_2001)).append("\n");
    sb.append("Jumlah Node Dieksplorasi :
").append(nodesExplored_2001).append("\n");

    resultArea_2001.setText(sb.toString());
}

// Method Wajib: displayGraph
private void displayGraph_2001() {
    panelGraph_2001.repaint();
}

// Method Wajib: resetGraph
private void resetGraph_2001() {
    resetState_2001();
    resultArea_2001.setText("Hasil Pencarian :
\nJalur :
\nNode
Dikunjungi :
\nJumlah Node Dieksplorasi : 0");
    displayGraph_2001();
}
}

```



```

private void resetState_2001() {
    visitedNodes_2001 = new ArrayList<>();
    pathNodes_2001 = new ArrayList<>();
    nodesExplored_2001 = 0;
}

// Inner Class untuk Gambar Graf
class GraphPanel_2001 extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        // Gambar Edge
        g2.setColor(Color.GRAY);
        g2.setStroke(new BasicStroke(2));
        for (String u : graph_2001.keySet()) {
            Point p1 = nodeCoords_2001.get(u);
            for (String v : graph_2001.get(u)) {
                Point p2 = nodeCoords_2001.get(v);
                g2.drawLine(p1.x, p1.y, p2.x, p2.y);
            }
        }

        // Gambar Node
        for (String node : nodeCoords_2001.keySet()) {
            Point p = nodeCoords_2001.get(node);

            // Logika Pewarnaan Node sesuai instruksi
            if (pathNodes_2001.contains(node)) {
                g2.setColor(Color.GREEN); // Jalur terpilih
            } else if (visitedNodes_2001.contains(node)) {
                g2.setColor(Color.ORANGE); // Node terekspansi
            } else {
                g2.setColor(Color.LIGHT_GRAY); // Belum disentuh
            }

            g2.fillOval(p.x - 15, p.y - 15, 30, 30);
            g2.setColor(Color.BLACK);
            g2.drawOval(p.x - 15, p.y - 15, 30, 30);

            // Nama Node
            g2.setFont(new Font("Arial", Font.BOLD, 12));
            g2.drawString(node, p.x - 20, p.y - 20);
        }
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {

```

```

        new PetaKampus_2511532001().setVisible(true);
    });
}
}

```

Peta yang dibangun merepresentasikan rancangan lokasi Unand dengan 10 simpul dan 15 sisi. Lokasi yang digunakan terdiri dari Gerbang, Rektorat, Masjid, Asrama, Perpustakaan, PKM, Fakultas Ekonomi, Fakultas Hukum, FMIPA, dan FTI. Struktur peta direpresentasikan menggunakan Adjacency List dengan `Map<String, List<String>>`, di mana setiap simpul menyimpan daftar lokasi yang terhubung dengannya. Peta ini bersifat tak berarah, sehingga setiap jalur dapat dilalui dua arah.

Pada program, BFS menelusuri graf secara melebar atau level-by-level menggunakan Queue. Algoritma ini mengecek semua tetangga dari node awal terlebih dahulu sebelum lanjut ke tingkat berikutnya, sehingga cocok digunakan untuk mencari jalur terpendek pada graf tanpa bobot. Sementara itu, DFS menelusuri graf secara mendalam menggunakan Stack. DFS akan memilih satu jalur terlebih dahulu hingga mentok, lalu melakukan backtrack untuk mencoba jalur lain. Karena itu, jalur yang dihasilkan DFS belum tentu yang paling pendek, melainkan jalur pertama yang berhasil mencapai tujuan. Kedua algoritma menggunakan HashMap untuk menyimpan asal tiap node, sehingga jalur dari tujuan ke titik awal dapat disusun kembali melalui fungsi `reconstructPath`.

3.2 Alasan Implementasi Algoritma

Ketiga algoritma tersebut diimplementasikan secara lengkap dalam satu program untuk menunjukkan kemampuan serta karakteristik unik dari masing-masing metode pengurutan tingkat lanjut. Shell Sort dimanfaatkan untuk mengoptimalkan pergeseran data secara alfabetis pada judul lagu tanpa perlu alokasi memori tambahan.

Quick Sort digunakan sebagai metode yang efisien untuk menangani perbandingan data numerik, khususnya pada atribut durasi lagu. Sementara itu, Merge Sort berperan sebagai penyeimbang karena memiliki sifat stabil, sehingga mampu menjaga konsistensi hasil pengurutan teks dalam berbagai kondisi.

BAB 4

KESIMPULAN

4.1 Ringkasan Hasil Praktikum

Praktikum ini memberikan pemahaman mengenai manipulasi struktur data berdimensi non-linear. Binary Tree efektif untuk menyimpan data hierarkis di mana setiap elemen induk memiliki percabangan spesifik. Kekuatan navigasi pada pohon biner sangat bergantung pada teknik rekursi yang dieksekusi melalui urutan Preorder, Inorder, dan Postorder. Sementara itu, Graph menawarkan arsitektur relasional yang jauh lebih fleksibel dibandingkan pohon karena tidak terikat pada hierarki root-leaf, melainkan berfokus pada topologi node dan edge. Navigasi pada jaringan graf membutuhkan pendekatan penandaan status memori (seperti HashSet) yang dipadukan dengan algoritma DFS untuk pencarian berbasis kedalaman jalur, atau BFS yang dibantu oleh Queue untuk pencarian berbasis radius jarak penyebaran antarsimpul.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Sorting Algorithms in Java". [Daring]. Tersedia pada:
<https://www.geeksforgeeks.org/sorting-algorithms/>. [Diakses: 3 Juni 2026].